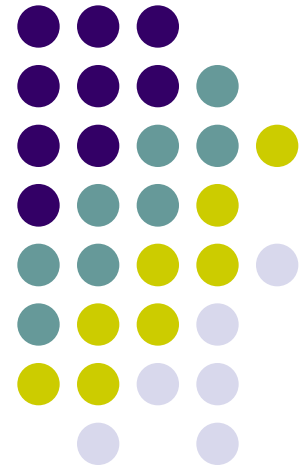
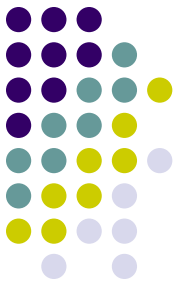


LẬP TRÌNH CĂN BẢN

Phần 2 - Chương 10 KIỂU TẬP TIN

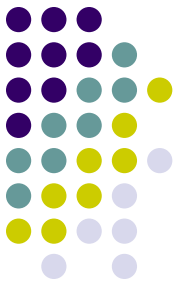




Nội dung chương này

- Một số khái niệm về tập tin
- Các thao tác trên tập tin
- Truy cập tập tin văn bản
- Truy cập tập tin nhị phân

Một số khái niệm về tập tin (file) (1)



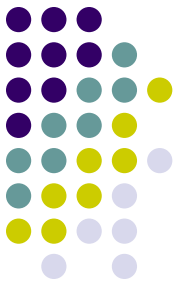
- Tại sao ta cần đến kiểu tập tin?
- Cho phép lưu trữ dữ liệu ở bộ nhớ ngoài (đĩa).
- Khi kết thúc chương trình thì dữ liệu vẫn còn do đó chúng ta có thể sử dụng nhiều lần.
- Kích thước lớn dữ liệu không hạn chế.

Một số khái niệm về tập tin (file) (2)

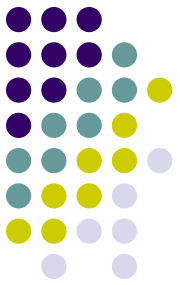


- Có 3 loại dữ liệu kiểu tập tin:
 - Tập tin văn bản (Text File)
 - Dùng để ghi các ký tự lên đĩa (dưới dạng mã Ascii)
 - Có chứa:
 - Ký hiệu ‘\n’ : xuống dòng
 - Ký tự EOF (End Of File) có mã Ascii là 26: nằm ở cuối tập tin
 - Tập tin định kiểu (Typed File)
 - Gồm nhiều phần tử có cùng kiểu: char, int, long, struct ...
 - Được lưu trữ trên đĩa dưới dạng một chuỗi các byte liên tục.
 - Tập tin không định kiểu (Untyped File)
 - Gồm các cấu trúc dữ liệu mà ta không quan tâm đến nội dung hoặc kiểu của nó.
 - Ta chỉ lưu ý đến các yếu tố vật lý của tập tin như độ lớn, ...

Một số khái niệm về tập tin (file)(3)

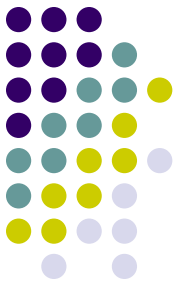


- **Biến tập tin**
 - Được dùng để đại diện cho một tập tin
 - Các thao tác lên tập tin sẽ được thực hiện thông qua biến này
- **Con trỏ tập tin**
 - Tại mỗi thời điểm, sẽ có một vị trí của tập tin mà tại đó việc đọc/ghi thông tin sẽ xảy ra
 - Ta hình dung có 1 con trỏ đang chỉ đến vị trí đó
 - Sau khi đọc/ghi xong dữ liệu, con trỏ sẽ chuyển dịch thêm một phần tử về phía cuối tập tin.
 - Sau phần tử dữ liệu cuối cùng của tập tin là dấu kết thúc tập tin EOF



Các thao tác trên tập tin

- Khai báo biến tập tin
- Mở tập tin
- Đóng tập tin
- Kiểm tra đến cuối tập tin hay chưa?
- Di chuyển con trỏ tập tin về đầu tập tin - Hàm `rewind()`



Khai báo biến tập tin

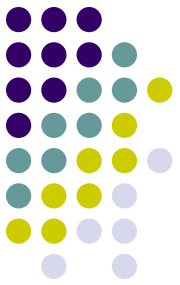
- **Cú pháp:**

FILE <Danh sách các biến con trỏ>;

- Các biến trong danh sách phải là các con trỏ và được phân cách bởi dấu phẩy(,).

- **Ví dụ:**

FILE *f1,*f2;



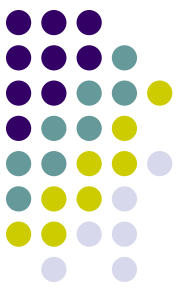
Mở tập tin (1)

- **Cú pháp:**

`FILE *fopen(char *Path, const char *Mode)`

- **Ý nghĩa:**

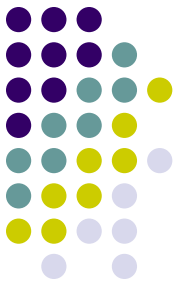
- Trả về con trỏ tập tin của tập tin được mở
- Trả về **NULL** nếu có lỗi



Mở tập tin (2)

- Path: chuỗi chỉ đường dẫn đến tập tin trên đĩa
- Type: chuỗi xác định cách thức mà tập tin sẽ mở. Các giá trị có thể của *Mode*:

Chế độ	Ý nghĩa
r	Mở tập tin văn bản để đọc
w	Tạo ra tập tin văn bản mới để ghi
a	Nối vào tập tin văn bản
rb	Mở tập tin nhị phân để đọc
wb	Tạo ra tập tin nhị phân để ghi
ab	Nối vào tập tin nhị phân
r+	Mở một tập tin văn bản để đọc/ghi
w+	Tạo ra tập tin văn bản để đọc ghi
a+	Nối vào hay tạo mới tập tin văn bản để đọc/ghi
r+b	Mở ra tập tin nhị phân để đọc/ghi
w+b	Tạo ra tập tin nhị phân để đọc/ghi
a+b	Nối vào hay tạo mới tập tin nhị phân



Mở tập tin (3)

- **Ví dụ:** Mở một tập tin tên TEST.txt để ghi.

```
FILE *f;
```

```
f = fopen("TEST.txt", "w");
```

```
if (f!=NULL){
```

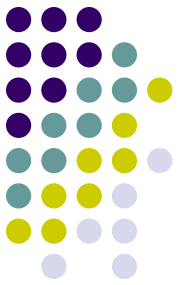
```
    // Các câu lệnh để thao tác với tập tin
```

```
    // Đóng tập tin
```

```
}
```

=> *mở tập tin để ghi*

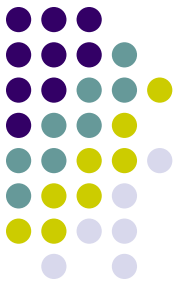
=> *nếu tập tin đã tồn tại rồi thì tập tin sẽ bị xóa và một tập tin mới được tạo ra*



Đóng tập tin

- **Cú pháp:** `int fclose(FILE *f)`
 - Ghi dữ liệu còn lại trong vùng đệm vào tập tin và đóng lại tập tin
 - `f` là con trỏ tập tin được mở bởi hàm `fopen()`
 - Giá trị trả về là 0 báo rằng việc đóng tập tin thành công
 - Giá trị trả về là `EOF` nếu có xuất hiện lỗi
- **Cú pháp:** `int fcloseall()`
 - Đóng tất cả các tập tin lại
 - Trả về tổng số các tập tin được đóng lại
 - Nếu không thành công, kết quả trả về là `EOF`

Kiểm tra đến cuối tập tin hay chưa?



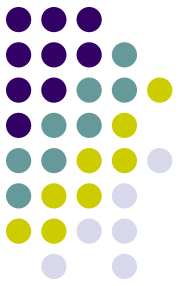
- **Cú pháp:**

```
int feof(FILE *f)
```

- **Ý nghĩa:**

- Kiểm tra xem đã chạm tới cuối tập tin hay chưa.
- Trả về EOF nếu cuối tập tin được chạm tới, ngược lại trả về 0.

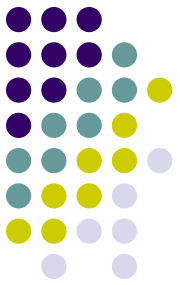
Di chuyển con trỏ tập tin về đầu tập tin - Hàm rewind()



- Cú pháp:

`void rewind(FILE *f)`

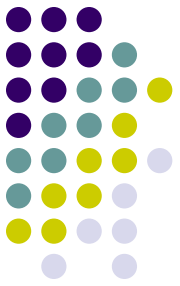
- Ý nghĩa:
 - Làm cho con trỏ quay về đầu tập tin như khi mở nó



Truy cập tập tin văn bản

- Ghi dữ liệu lên tập tin văn bản
- Đọc dữ liệu từ tập tin văn bản

Ghi dữ liệu lên tập tin văn bản (1)

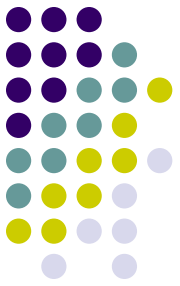


- **Hàm putc()**

`int putc(int c, FILE *f)`

- Được dùng để ghi một ký tự lên một tập tin văn bản đang được mở (liên kết với con trỏ f) để làm việc
- `c` chứa mã **Ascii** của ký tự
- Hàm này trả về **EOF** nếu gặp lỗi

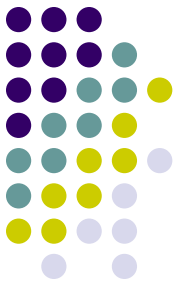
Ghi dữ liệu lên tập tin văn bản (2)



- **Hàm fputs()**

`int fputs(const char *buffer, FILE *f)`

- Được dùng để ghi một chuỗi ký tự chứa trong vùng đệm lên tập tin văn bản
- Hàm này trả về giá trị **0** nếu **buffer** chứa chuỗi rỗng và trả về **EOF** nếu gặp lỗi

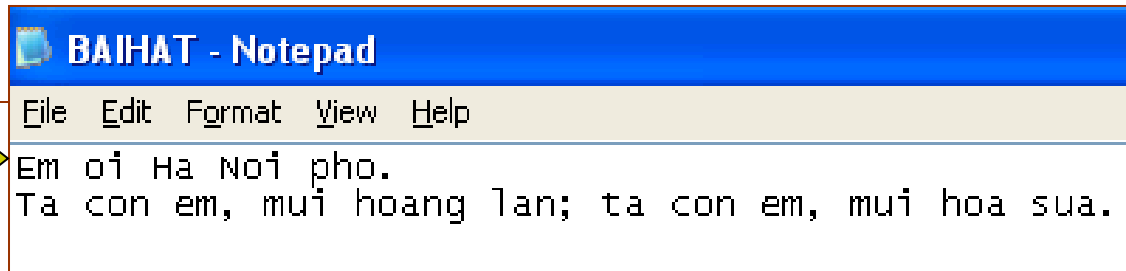


Ghi dữ liệu lên tập tin văn bản (3)

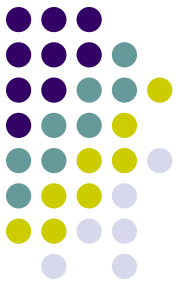
- **Ví dụ:** Viết chương trình ghi chuỗi ký tự lên tập tin văn bản **D:\\Baihat.txt**

```
#include<stdio.h>
#include<conio.h>
int main(){
    FILE *f;
    clrscr();
    f=fopen("D:\\Baihat.txt","w+");

    if (f!=NULL){
        fputs("Em oi Ha Noi pho.\n",f);
        fputs("Ta con em, mui hoang lan; ta con em, mui hoa sua.",f);
        fclose(f);
    }
    getch();
    return 0;
}
```



Ghi dữ liệu lên tập tin văn bản (3)

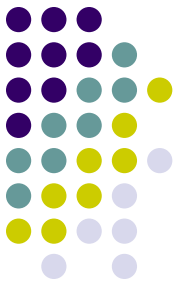


- **Hàm fprintf()**

`fprintf(FILE *f, const char *format, varexpr)`

- Được dùng để ghi dữ liệu có định dạng lên tập tin văn bản.
- `format`: chuỗi định dạng (giống với các định dạng của hàm `printf()`)
- `varexpr`: danh sách các biểu thức, mỗi biểu thức cách nhau dấu phẩy (,)

Đọc dữ liệu từ tập tin văn bản (1)

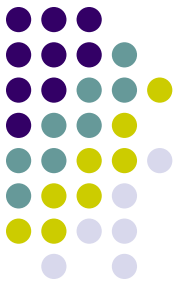


- **Hàm getc()**

`int getc(FILE *f)`

- Được dùng để đọc dữ liệu từ tập tin văn bản đang được mở để làm việc (liên kết với f)
- Hàm này trả về mã Ascii của một ký tự được đọc (kể cả EOF)

Đọc dữ liệu từ tập tin văn bản (2)

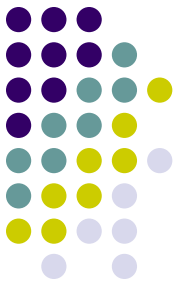


- **Hàm fgets()**

`char *fgets(char *buffer, int n, FILE *f)`

- Được dùng để đọc 1 chuỗi ký tự từ tập tin văn bản đang được mở (liên kết với con trỏ f)
- Đọc cho đến khi đủ n ký tự hoặc gặp ký tự xuống dòng ‘\n’ (ký tự này cũng được đưa vào chuỗi kết quả) hay gặp ký tự kết thúc EOF (ký tự này không được đưa vào chuỗi kết quả)
- buffer: chỉ đến cùng nhớ đủ lớn chứa các ký tự nhận được
- Ký tự NULL (‘\0’) tự động được thêm vào cuối chuỗi kết quả lưu trong vùng đệm
- Hàm trả về địa chỉ đầu tiên của vùng đệm khi không gặp lỗi và chưa gặp ký tự kết thúc EOF. Ngược lại, hàm trả về giá trị NULL

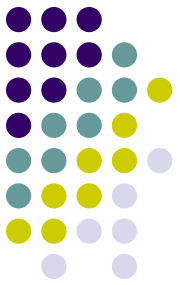
Đọc dữ liệu từ tập tin văn bản (3)



- **Hàm fscanf()**

fscanf(**FILE** *f, **const char** *format, varlist)

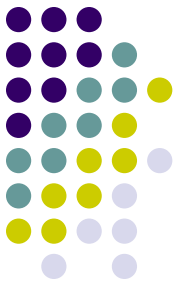
- Được dùng để đọc dữ liệu từ tập tin văn bản vào danh sách các biến theo định dạng.
- format: chuỗi định dạng (giống hàm scanf())
- varlist: danh sách các biến mỗi biến cách nhau dấu phẩy (,).



Đọc dữ liệu từ tập tin văn bản (4)

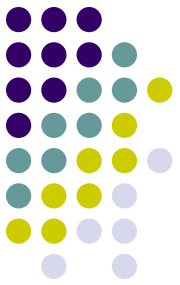
- **Ví dụ:** Viết chương trình chép tập tin `D:\Baihat.txt` ở trên sang tập tin `D:\Baica.txt`.

```
#include<stdio.h>
#include<conio.h>
int main(){
    FILE *f1,*f2;
    clrscr();
    f1=fopen("D:\\Baihat.txt","rt");
    f2=fopen("D:\\Baica.txt","wt");
    if (f1!=NULL && f2!=NULL){
        int ch=fgetc(f1);
        while (!feof(f1)){
            fputc(ch,f2);
            ch=fgetc(f1);
        }
        fcloseall();
    }
    getch();
    return 0;
}
```



Truy cập tập tin nhị phân

- Ghi dữ liệu lên tập tin nhị phân
- Đọc dữ liệu từ tập tin nhị phân
- Di chuyển con trỏ tập tin
- Ví dụ

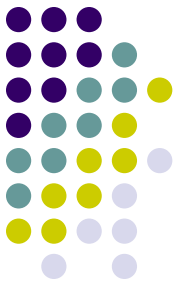


Ghi dữ liệu lên tập tin nhị phân

- **Hàm fwrite()**

`size_t fwrite(const void *ptr, size_t size, size_t n, FILE *f)`

- **ptr**: con trỏ chỉ đến vùng nhớ chứa thông tin cần ghi lên tập tin.
- **n**: số phần tử sẽ ghi lên tập tin.
- **size**: kích thước của mỗi phần tử.
- **f**: con trỏ tập tin đã được mở.
- Giá trị trả về của hàm này là số phần tử được ghi lên tập tin. Giá trị này bằng n trừ khi xuất hiện lỗi.

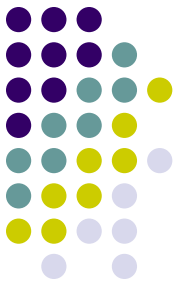


Đọc dữ liệu từ tập tin nhị phân

- **Hàm fread()**

size_t fread(const void *ptr, size_t size, size_t n, FILE *f)

- ptr: con trỏ chỉ đến vùng nhớ sẽ nhận dữ liệu từ tập tin
- n: số phần tử được đọc từ tập tin
- size: kích thước của mỗi phần tử
- f: con trỏ tập tin đã được mở
- Giá trị trả về của hàm này là số phần tử đã đọc được từ tập tin. Giá trị này bằng n hay nhỏ hơn n nếu đã chạm đến cuối tập tin hoặc có lỗi xuất hiện



Di chuyển con trỏ tập tin

- Hàm fseek()

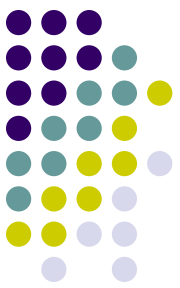
`int` fseek(`FILE` *f, `long` offset, `int` whence)

- Được dùng để di chuyển con trỏ tập tin đến vị trí chỉ định
- f: con trỏ tập tin đang thao tác
- offset: số byte cần dịch chuyển con trỏ tập tin kể từ vị trí trước đó. Phần tử đầu tiên là vị trí 0.
- whence: vị trí bắt đầu để tính offset, ta có thể chọn điểm xuất phát là

0	SEEK_SET	Vị trí đầu tập tin
1	SEEK_CUR	Vị trí hiện tại của con trỏ tập tin
2	SEEK_END	Vị trí cuối tập tin

- Kết quả trả về của hàm là 0 nếu việc di chuyển thành công. Nếu không thành công, 1 giá trị khác 0 (đó là 1 mã lỗi) được trả về.

Ví dụ



- Viết chương trình ghi lên tập tin CacSo.Dat 3 giá trị số (thực, nguyên, nguyên dài). Sau đó đọc các số từ tập tin vừa ghi và hiển thị lên màn hình

```
#include<stdio.h>
#include<conio.h>
int main(){
    clrscr();
    FILE *f;
    f=fopen("D:\\CacSo.txt", "wb");
    if (f!=NULL){
        double d=3.14;
        int i=101;
        long l=54321;
        fwrite(&d,sizeof(double),1,f);
        fwrite(&i,sizeof(int),1,f);
        fwrite(&l,sizeof(long),1,f);
```

```
        // Doc tu tap tin
        rewind(f);
        fread(&d,sizeof(double),1,f);
        fread(&i,sizeof(int),1,f);
        fread(&l,sizeof(long),1,f);
        printf("Cac ket qua la: %f %d %ld",d,i,l);
        fclose(f);
    }
    getch();
    return 0;
}
```

Hết chương

